

# DevSecOps, explained from first principles

Understand the flow. Understand the findings. Make a defensible release decision.

## FOUNDATIONS

What each stage does, why it exists, and the terms you need.

## DETECTION

Inputs, working principles, comparison data, updates and blind spots.

## IMPLEMENTATION

Repository commands, real gate behavior, evidence and release integration.

**Three learning paths, one consistent explanation.**

# Choose a path through the material

Read the main course in sequence; return to technical sections when implementing.

## BEGINNER

Start with foundations and the four pipeline phases. Read every term and worked example.

## INTERMEDIATE

Focus on detection, database freshness, command behavior and troubleshooting.

## PROFESSIONAL

Review trust boundaries, evidence integrity, exceptions and the release contract.

**A scanner finding, a policy decision and a production release are different events.**

# The problem: one change can carry many risks

Example: a developer changes checkout in an online shop.

## SOURCE

A token enters a file. A database query uses unsafe input. A new package brings old dependencies.

## DELIVERABLE

The container includes an outdated system library. Its deployment requests excessive privilege.

## RUNNING SERVICE

One customer can access another customer's order. A compromised process starts a shell.

**Choose controls according to the risk and the object being inspected.**

# The delivery vocabulary

Expand the terms before reading a pipeline diagram.

## CI / CD

Continuous Integration: merge and verify changes frequently.  
Continuous Delivery: keep releases ready for approval.

## PR / BUILD

Pull Request: propose a reviewed change. Build: turn source and dependencies into a distributable artifact.

## STAGE / RELEASE

Staging: a controlled test environment. Release: approve a specific artifact for use.

**This course uses CD to mean Continuous Delivery with an explicit release decision.**

# The security vocabulary

Each acronym names a different activity.

## SCA / SAST

Software Composition Analysis: inspect dependencies. Static Application Security Testing: inspect code without running the app.

## IaC / DAST

Infrastructure as Code: machine-readable infrastructure. Dynamic Application Security Testing: inspect a running service.

## SBOM / GATE

Software Bill of Materials: component inventory. Security gate: a rule deciding whether work may continue.

**Tool names are products; SCA, SAST and DAST are categories of work.**

# A vulnerability ID is not a risk score

CVE, CWE and CVSS answer different questions.

## CVE

Common Vulnerabilities and Exposures: an identifier for a disclosed vulnerability. An advisory may also have a GHSA or OSV ID.

## CWE

Common Weakness Enumeration: a class of mistake, such as improper input handling.

## CVSS

Common Vulnerability Scoring System: a severity framework. Keep its version, score and vector together.

**No CVE does not mean no security issue.**

# Prioritize with evidence and context

Severity, exploitation and business impact are separate inputs.

## CVSS

How severe can exploitation be?  
Common bands: low 0.1–3.9,  
medium 4.0–6.9, high 7.0–8.9,  
critical 9.0–10.

## EPSS / KEV

Exploit Prediction Scoring  
System estimates exploitation  
probability. Known Exploited  
Vulnerabilities records observed  
exploitation.

## YOUR SERVICE

Is the component shipped,  
reachable and exposed? What  
data or operations can an  
attacker affect?

**A low prediction is not an exemption; a score needs application context.**

# The complete flow in four phases

Each phase produces evidence needed by the next.

## 1 PREPARE

Requirements → threat  
model → reviewed  
source



## 2 VERIFY

Secrets +  
dependencies + code +  
configuration + tests



## 3 PACKAGE

Build → candidate  
registry → image scan  
+ inventory + trust



## 4 RELEASE

Staging → behavior  
tests → gate →  
production → feedback

**The diagram expresses dependencies; it does not require every scan to run serially.**

# Phase 1: define what must be protected

Owner: product team with security and platform engineering.

## INPUT

Data, users, trust boundaries, critical business actions and architecture.

## WORK

Write abuse cases. Define access rules, recovery needs and acceptance tests.

## OUTPUT / GATE

Reviewed requirements, a threat model and named owners. Unresolved critical design risks need a decision.

**Requirements become tests; a vague security goal cannot be verified.**

## Phase 2: verify the proposed change

Owner: developers; security engineers maintain rules and triage support.

### CHECKOUT

Obtain the intended commit.  
Protect reviews, pipeline  
changes and runner credentials.

### ANALYZE

Run secret, dependency, code  
and configuration checks. Run  
unit, integration and security  
tests.

### DECIDE

Fail on configured violations or  
tool errors. Save reports and  
correct the change before  
retrying.

**A report file existing is not proof that the scanner completed or found nothing.**

## Phase 3: identify and verify the artifact

Owner: build and platform engineering.

### BUILD ONCE

Produce one candidate image from the reviewed source and controlled dependencies.

### INSPECT

Scan the image by digest. Generate its component inventory. Record build provenance.

### ESTABLISH TRUST

Sign the digest and relevant attestations. Store evidence in a restricted system.

**Source scans and image scans inspect different inputs; retain both sets of evidence.**

## Phase 4: test, release and observe

Owner: application team and release operations.

### STAGING

Deploy the candidate digest.  
Check health, authenticated  
behavior and API access rules.

### RELEASE GATE

Verify identity, complete reports,  
thresholds, valid exceptions and  
approval.

### PRODUCTION

Deploy the approved digest.  
Observe health and security; roll  
back or patch when needed.

**A passing release gate is a time-bounded decision about one artifact.**

# Five working principles behind the tools

Ask what is being compared before asking which database is used.

## IDENTITY + ADVISORY

SCA and image scanners identify packages and compare versions with affected ranges.

## STRUCTURE + RULE

Secret, code and configuration tools match patterns, data flows or policy conditions.

## OBSERVATION + TRUST

DAST inspects responses; runtime tools inspect events; signing tools verify cryptographic evidence.

**“Database” can mean advisories, a code model, an inventory or a rule bundle.**

# Secret scanning: Gitleaks

Find credential-shaped material before it spreads.

## INPUT / FINDINGS

Files and Git history. Finds likely tokens, keys and other secrets with file and rule context.

## HOW IT WORKS

Rules use patterns, keywords and optional entropy checks. Entropy measures apparent randomness.

## DATA / LIMITS

Embedded and custom rules; no CVE feed. A pattern match does not establish that a credential is active.

**Rotate a real leaked credential; deleting the string does not invalidate it.**

# SCA: identify, then match

Software Composition Analysis starts with an accurate inventory.

## IDENTIFY

Read supported lockfiles,  
package metadata or binaries.  
Resolve name, ecosystem,  
version and origin.

## COMPARE

Select the relevant advisory  
source. Evaluate affected  
version ranges using  
ecosystem-specific semantics.

## REPORT

Return component, advisory ID,  
severity, installed version, fixed  
version and evidence path.

**A missed component cannot receive a reliable vulnerability verdict.**

# Worked example: an affected version range

Illustrative package and advisory; these are not real vulnerability records.

## INVENTORY

The lockfile resolves demo-parser 2.4.1. A transitive dependency brought it into the shop.

## ADVISORY

The relevant ecosystem advisory says versions from 2.0.0 up to, but excluding, 2.4.3 are affected.

## DECISION

2.4.1 is inside the range.  
Upgrade resolution to a tested fixed release, rebuild and rescan.

**Name + ecosystem + version + origin determine which advisory applies.**

# Trivy: source and image vulnerabilities

One product, several scanners; enable the intended scope explicitly.

## INPUT

Filesystem manifests or image package metadata. An image adds operating-system packages and packaged application libraries.

## COMPARISON DATA

Trivy DB aggregates vendor and ecosystem advisories. OS-managed packages use the corresponding distribution source.

## OUTPUT / LIMIT

Known vulnerable components. This project enables vulnerability scanning; licenses and image secrets need separate controls.

**Check which scanner ran and which packages it recognized.**

# Dependency-Check, OSV and Gype

Alternative matching systems share a goal but differ in identity and data.

## DEPENDENCY-CHECK

Analyzers collect package evidence, identify CPE candidates and consult NVD applicability data.

## OSV

Open Source Vulnerabilities uses ecosystem-oriented records, package versions and affected ranges or commits.

## GRYPE

Matches inventoried packages against its vulnerability database; can scan an image, filesystem or supported SBOM.

**Compare evidence and coverage when tools disagree; do not just count alerts.**

# SAST: follow data through code

Static Application Security Testing looks for unsafe program behavior.

## SOURCE

Untrusted data enters through a request parameter, file, message or other boundary.

## FLOW

Analysis follows values through assignments and calls. Models describe sanitizers and safe transformations.

## SINK

A sensitive operation uses the value: a database query, command execution or rendered page.

**Inspect the source-to-sink evidence before deciding whether the finding is real.**

# Semgrep: executable rules over code

Rules define what the engine should recognize.

## DETECTION

Structural patterns and taint rules describe unsafe code. Analysis depth depends on engine, language and edition.

## DATA / UPDATES

Local YAML rules or registry rulesets. Pin and review rules; upgrade the engine separately.

## PROJECT BEHAVIOR

The local demonstration rule flags ignored pipeline failures. `--error` makes findings fail; `--strict` handles scan errors more strictly.

**A successful process can contain findings unless failure behavior is configured.**

# CodeQL and SonarQube use different models

A database can represent your code rather than public vulnerabilities.

## CODEQL

Extracts a database representing source. Security queries analyze the code model and may return data-flow paths.

## SONARQUBE

Analyzers apply language rules. Quality profiles choose active rules; quality gates evaluate configured metrics.

## INTEGRATION

Match analyzer support to the language and build. Retrieve the final gate result; scanner upload success is insufficient.

**Confirm analysis completion and the evaluated policy, not just report upload.**

# IaC scanning: compare desired state to policy

Infrastructure as Code can create exposure before the application starts.

## FINDS

Examples include public storage, broad network access, privileged workloads and missing encryption settings.

## HOW

Parse configuration and evaluate resource attributes or relationships against checks and organizational policy.

## DATA / LIMITS

Rules and policy bundles, not primarily a CVE database. Templates may not equal deployed state.

**Configuration scanning predicts risk in declared infrastructure; verify deployed reality too.**

# Containers: inspect what actually ships

An application can be clean while its base image is vulnerable.

## LAYERS

A container image packages application files, libraries and often operating-system packages.

## SCAN TARGET

Inspect the release digest for the intended platform. Include base image and application package metadata.

## HARDENING

Use minimal trusted bases, non-root execution, limited permissions and controlled updates.

**An image scan is not a runtime monitor or a complete hardening review.**

# SBOM: inventory for today and tomorrow

Software Bill of Materials records component identity and relationships.

## GENERATE

Syft or Trivy examines the artifact and emits a structured inventory.

## EXCHANGE

CycloneDX and SPDX are interoperable formats. SPDX expands to Software Package Data Exchange.

## USE

Link the inventory to the artifact digest. Recheck it when new vulnerability information appears.

**An inventory enables vulnerability analysis; it is not a clean bill of health.**

# Signing and provenance establish origin

Verify both cryptographic integrity and the expected identity.

## DIGEST / SIGNATURE

The digest identifies content. A signature binds a signing key or identity to that content.

## PROVENANCE

An attestation describes the builder and inputs. Its trust depends on the builder and how evidence was produced.

## VERIFY

Check the expected signer, identity issuer, artifact digest and required claims before deployment.

**Signed does not mean safe; verified expected origin is one release condition.**

# DAST: inspect the running application

Dynamic Application Security Testing observes requests and responses.

## DISCOVER

Find routes through crawling, recorded traffic or an API definition. Establish the intended authenticated session.

## EXERCISE

Observe traffic or send targeted test inputs. Evaluate responses, behavior and rule-specific evidence.

## REPORT

Record endpoint, parameter, evidence, rule, confidence and coverage. Reproduce important findings safely.

**A clean scan of a login page says little about the authenticated application.**

# ZAP: baseline is not a full active test

ZAP rules arrive through the installed tool and add-ons.

## BASELINE

A short crawl followed by passive checks. Useful for headers and observable response issues.

## FULL SCAN

Discovery plus active testing. Use controlled scope, test data, rate limits and an authorized environment.

## EXIT STATUS

Packaged scans distinguish FAIL, WARN and execution errors. Choose an explicit handling policy for each.

**State the scan mode, authenticated coverage and outcome in every DAST report.**

# API security needs business-aware assertions

Example: order access must remain within a customer account.

## TEST IDENTITY

Create customer A, customer B and an administrative role using isolated test accounts.

## TEST OWNERSHIP

Request A's order as B. Expect access denial and no order data, including through alternate routes.

## TEST BOUNDARIES

Check role changes, pagination, bulk endpoints, expired sessions and rate-sensitive operations.

**Two authenticated users are often more useful than one anonymous scan.**

# Admission control: prevent a bad deployment

Kubernetes admission evaluates requests before the cluster accepts them.

## INPUT

A workload manifest or deployment request, plus relevant policy and context.

## COMPARE

Kyverno or Open Policy Agent rules test settings and, when configured, image trust requirements.

## MODE MATTERS

Audit records violations. Enforce rejects violating requests. The project's sample is Audit mode.

**A policy file in the repository does nothing until it is installed and enforced.**

# Runtime detection: notice suspicious behavior

Falco inspects events after software starts running.

## OBSERVE

System activity and supported event sources produce process, file and other runtime context.

## MATCH

Rules compare event fields and conditions with behaviors worth investigating.

## RESPOND

Route alerts to an owner. Investigate, contain when justified, preserve evidence and correct the cause.

**Detection requires an operational response path; an unowned alert is incomplete.**

# How a disclosure becomes a scanner finding

There are several independent delays between discovery and detection.

## DISCLOSE

Researchers and vendors investigate. An advisory and identifier become public.



## ENRICH

Maintainers and databases add package ranges, severity, references and applicability.



## DISTRIBUTE

A scanner provider builds or exposes updated intelligence. Your runner refreshes its local data.



## MATCH

The next scan inventories your artifact and evaluates it against that snapshot.

**Discovery time, public disclosure, database refresh and your next scan are different clocks.**

# When Trivy updates its data

Verified upstream schedules; actual local freshness depends on metadata and execution.

## VULNERABILITY DB

Upstream publication is scheduled every 6 hours. Successful publication can be delayed by outages.

## JAVA INDEX

A separate package-identification index has a daily publication schedule. It is not a second CVE feed.

## LOCAL RUNNER

Refresh depends on cache metadata, execution and flags. Record UpdatedAt, NextUpdate and DownloadedAt where available.

**Measure the database actually used, not just the provider's nominal schedule.**

# Refresh intervals across other tools

Some tools refresh data; others need a new rule pack or executable.

## DEPENDENCY-CHECK

The documented default NVD recheck interval is 4 hours. A scan or update job must actually run.

## RULE-BASED TOOLS

Gitleaks, SAST, IaC and ZAP rules change through tool, policy or add-on updates. There is no common CVE refresh timer.

## PRIORITIZATION

EPSS is refreshed daily. KEV grows as qualifying evidence is added. Poll and preserve the received snapshot.

**Record both tool version and intelligence or rules revision.**

# Freshness is an operational control

Suggested operating policy, not a product default or universal requirement.

## BEFORE A SCAN

Check access to the approved source. Refresh as needed. Retain version, timestamps and update logs.

## ON AN OUTAGE

Distinguish stale data from no findings. Block release when the accepted freshness budget is exceeded.

## AFTER RELEASE

Schedule rescans of supported production artifacts. Trigger urgent checks after relevant new advisories.

**No update + no finding can still mean “insufficient evidence.”**

# How to read a vulnerability report

Follow one result from package identity to remediation.

## IDENTITY

Target and digest; package name, ecosystem, installed version and evidence path.

## ADVISORY

Identifier, affected range, severity source, fixed version and reference links.

## CONTEXT

Tool and database versions, scan time, scope, suppressions and whether the package is shipped.

**A useful finding explains what was matched, where, why and what to do next.**

# False positives, false negatives and unknowns

Accuracy includes knowing the limits of the inspection.

## FALSE POSITIVE

A reported issue does not apply: for example, incorrect package identity or a valid vendor backport.

## FALSE NEGATIVE

A real issue is missed: for example, an unsupported file or missing authenticated route.

## UNKNOWN

Evidence is incomplete: for example, a failed refresh, unresolved version or parser error.

**Reduce noise through evidence and scoped rules, not broad exclusions.**

# Turn scanner output into an explicit gate

This project uses a conservative, simple executable baseline.

## SOURCE / IMAGE

Trivy blocks HIGH and CRITICAL findings. Gitleaks blocks configured matches. Semgrep blocks local-rule findings.

## ERRORS

Missing tools and failed scans stop their wrappers. Failure statuses propagate to the caller.

## RELEASE

Unconfigured DAST and release verification stop with exit 2. They cannot establish production readiness.

**Document implemented behavior separately from the desired production architecture.**

# The release decision must fail closed

Missing, stale or mismatched evidence prevents approval.

## BIND

Match the commit, build run, platform and artifact digest across all required evidence.

## VERIFY

Check required scan completion, coverage, freshness, signatures, provenance and trusted issuer identity.

## EVALUATE

Apply versioned policy and validated exceptions. Save the reasons and the approved digest.

**A gate that always prints success is a missing control.**

# Exceptions need a lifecycle

An exception is a documented risk decision with a limit.

## REQUEST

Identify finding, affected artifact or scope, reason, impact and proposed mitigation.

## APPROVE

Assign an accountable approver, owner, expiry and remediation ticket. Protect approval records.

## REVALIDATE

Check the scope and UTC expiry on each release. Reject expired or altered approvals.

**An exception does not remove the finding or the obligation to remediate it.**

# What belongs in the release evidence package

Keep enough context to repeat and defend the decision.

## SOURCE + TESTS

Commit, reviewed change, runner/build identity, test results, scan scope and completion status.

## ARTIFACT + DATA

Digest and platform, SBOM, reports, tool versions, rule revisions and advisory snapshot metadata.

## DECISION + DEPLOY

Policy version, verified trust, approvals, exceptions, target environment and deployed digest.

**Evidence is useful only when its origin, completeness and artifact binding can be trusted.**

# What this repository actually provides

A teaching reference with runnable wrappers and incomplete deployment integrations.

## AVAILABLE

Source scanner wrappers, image and SBOM commands, signing entry point, policies and CI examples.

## DEMONSTRATION

One Semgrep rule; an Audit-mode Kubernetes policy; illustrative reports and stack notes.

## INTEGRATION REQUIRED

Tool provisioning, real application tests/build, DAST, trust verification, exceptions and deployment.

**Use the implementation guide to turn the reference into a tested application pipeline.**

# Local use: prerequisites and first checks

Run Bash wrappers in Git Bash, Linux or a configured WSL environment.

## PREPARE

Install reviewed Git, Gitleaks, Trivy and Semgrep versions.  
Allow required advisory access.

## EXECUTE

```
bash scripts/preflight.sh
bash scripts/secret-scan.sh
bash scripts/sca-scan.sh
```

## CONTINUE

```
bash scripts/sast-scan.sh
bash scripts/iac-scan.sh
Inspect reports/generated/ and
exit statuses.
```

See [docs/implementation/quickstart.md](#) for commands, outputs and troubleshooting.

# A stack changes inputs, not the control goals

Use the same reasoning across application languages.

## NODE / JAVA

Node: resolved package lock and final image. Java: resolved Maven or Gradle graph, packaged archives and runtime image.

## PYTHON / .NET

Python: pinned resolution plus built wheel/image. .NET: resolved NuGet dependencies and published application.

## GO

Module resolution plus built binary metadata and image. Add language-aware vulnerability and code checks as needed.

**Scan the resolved dependencies and the final deliverable; validate the supported formats.**

# CI templates need explicit setup

Continuous Integration platforms differ in syntax and trust configuration.

## RUNNER

Provision reviewed tool versions.  
Isolate untrusted changes. Use  
least-privilege, short-lived  
credentials.

## JOBS

Preserve scanner failures and  
always collect reports. Pass the  
artifact digest explicitly between  
jobs.

## PROMOTION

Require checks and protected  
environments. Verify policy  
before production deployment.

**A copied YAML file is an integration starting point, not proof of enforcement.**

# Corrections made during this review

Changes address misleading pass results and inconsistent scope.

## SCAN WRAPPERS

Explicit configuration, blocking exit behavior, worktree secret coverage and digest validation.

## PIPELINES

Removed ignored scan failures. Fixed chained command failure handling. Added clearer prerequisite and evidence guidance.

## LEARNING MATERIAL

Rebuilt flow, expanded terminology, explained comparison data and separated implementation from target design.

See [docs/PROJECT\\_REVIEW.md](#) for the issue list, changes and remaining integration work.

# Troubleshooting by symptom

Resolve missing evidence before interpreting security results.

## TOOL NOT FOUND

Check PATH and runner provisioning. Preflight or the wrapper should fail instead of silently skipping.

## DB DOWNLOAD FAILS

Check network, proxy, rate limits, credentials and supported cache format. Review the last successful update.

## ZERO FINDINGS

Check recognized files, packages, rules, exclusions and authenticated routes before calling it clean.

**“The tool ran” and “the intended scope was assessed” are separate checks.**

# After release: a new CVE changes the answer

The artifact is unchanged; the available knowledge has changed.

## DETECT

A new advisory or KEV entry arrives. Search inventories and rescan affected supported artifacts.

## RESPOND

Confirm applicability and exposure. Assign an owner, fix or mitigate, and set a response deadline.

## RELEASE AGAIN

Rebuild from controlled inputs, repeat the affected checks, deploy and verify the new digest.

**A security result expires as the artifact, environment or intelligence changes.**

# Standards guide outcomes and assurance

This architecture is a reference; its exact sequence is not universally mandated.

## NIST SSDF

Secure Software Development Framework: organizational practices for preparing, protecting, producing and responding.

## OWASP

Open Worldwide Application Security Project: application requirements, verification guidance and maturity resources.

## SLSA

Supply-chain Levels for Software Artifacts: requirements and assurance concepts for source and build integrity.

**Use named versions and evidence-based mappings; avoid unsupported compliance claims.**

# Adopt in small, verifiable increments

Suggested engineering sequence; adapt it to application risk.

## FOUNDATION

Protect source, provision tools, run tests and enforce a small set of high-signal source checks.

## ARTIFACT TRUST

Scan the deliverable, generate inventory, verify signing and preserve the same digest through staging.

## OPERATIONS

Add authenticated behavior tests, trusted release evidence, admission controls and recurring response.

**One reliable, owned control is more useful than several unverified report generators.**

# Check your understanding

Try answering before opening the explanations in the notes.

## QUESTION 1

Why can a new CVE appear in an image that has not changed?

## QUESTION 2

Why does a clean ZAP baseline not prove that customer authorization is correct?

## QUESTION 3

Why must release verification reject a report about the wrong digest?

**If you can explain the evidence behind a decision, you understand the pipeline.**

# What a defensible release can explain

Five questions connect the entire course.

## WHAT CHANGED?

Identify reviewed source, dependencies and build inputs.

## WHAT WAS CHECKED?

Show the scope, outcomes, tool/rule/data versions and remaining uncertainty.

## WHAT WAS RELEASED?

Show the approved digest, trusted origin, policy decision and production deployment record.

**Reviewed source → tested artifact → verified release → continuous response.**

# Sources and maintenance

Official documentation; full clickable links are in the slide notes and handbook.

## PRIMARY REFERENCES

CVE identifier lifecycle

NVD vulnerability enrichment

## PRIMARY REFERENCES

Trivy: vulnerability matching

Trivy database build and metadata

## PRIMARY REFERENCES

Trivy database publication schedule

Trivy Java index

**Source review date: 2026-09-16. Local configuration determines actual behavior.**

# Sources and maintenance

Official documentation; full clickable links are in the slide notes and handbook.

## PRIMARY REFERENCES

Java index publication schedule

Trivy database refresh controls

## PRIMARY REFERENCES

Dependency-Check detection  
model

Dependency-Check update  
options

## PRIMARY REFERENCES

Gitleaks detection and  
configuration

Semgrep rule and analysis  
concepts

**Source review date: 2026-09-16. Local configuration determines actual behavior.**

# Sources and maintenance

Official documentation; full clickable links are in the slide notes and handbook.

## PRIMARY REFERENCES

Semgrep command and exit semantics

CodeQL database and query model

## PRIMARY REFERENCES

SonarQube analysis overview

Checkov configuration and graph checks

## PRIMARY REFERENCES

ZAP baseline scope and exit codes

ZAP full scan

**Source review date: 2026-09-16. Local configuration determines actual behavior.**

# Sources and maintenance

Official documentation; full clickable links are in the slide notes and handbook.

## PRIMARY REFERENCES

ZAP add-ons and rules

Syft software inventory

## PRIMARY REFERENCES

Grype vulnerability scanning

Cosign signature verification

## PRIMARY REFERENCES

Kyverno admission validation

Falco runtime rules

**Source review date: 2026-09-16. Local configuration determines actual behavior.**

# Sources and maintenance

Official documentation; full clickable links are in the slide notes and handbook.

## PRIMARY REFERENCES

FIRST Exploit Prediction Scoring System

FIRST CVSS v4.0 specification

## PRIMARY REFERENCES

CISA official KEV data mirror

MITRE Common Weakness Enumeration

## PRIMARY REFERENCES

Open Source Vulnerabilities

NIST SP 800-218 SSDF v1.1

**Source review date: 2026-09-16. Local configuration determines actual behavior.**

# Sources and maintenance

Official documentation; full clickable links are in the slide notes and handbook.

## PRIMARY REFERENCES

SLSA v1.2 specification

**Source review date: 2026-09-16. Local configuration determines actual behavior.**